Owner Action Plan — 20 Projects

For Siddhant Singh. Prepared 2026-10-14. Every step here needs your hardware, your accounts, your signature or your judgement, so an agent cannot do it. Each step says where to click or what to type, what success looks like, what to save, and what to do if it fails.

How to read this. Start with the one-page checklist (section 1). Section 2 maps each project to its owner-only steps. Sections 3 onward are the procedures. PowerShell commands run in Windows ("PowerShell" blocks). Linux commands run inside WSL2 Ubuntu ("WSL" blocks). Paths like `~/src/PROJECTS` are suggestions; nothing here uses an account-specific path or a secret. Wherever a step says "save", put the file in the project's `docs/evidence/owner/` folder and commit it.

Contents

1. One-page prioritized checklist 2

2. Project-to-owner-step matrix 3

3. Laptop setup: RTX 4060 + WSL2 + CUDA + Docker 4

3.1 Confirm the operating system . . . 4

3.2 Install WSL2 with Ubuntu 24.04 . . . 4

3.3 Install the NVIDIA Windows driver (the only GPU driver you need) . . . 4

3.4 Install the CUDA toolkit inside WSL (compiler, libraries) . . . 4

3.5 Install Docker Desktop with GPU support . . . 5

3.6 Get the code and run the one-command diagnostic . . . 5

3.7 Rules for every GPU measurement . . . 5

4. Benchmark evidence capture (applies to every project) 6

5. GPU and laptop runs, project by project 7

5.1 CUDA kernels (RTX 4060, sm_89): about 30 minutes . . . 7

5.2 Nebula GPU cloud: a real GPU job in a container, about 30 minutes . . . 7

5.3 Distributed training: single-GPU NCCL check, about 20 minutes . . . 7

5.4 LLM inference engine and InferX: no GPU step . . . 8

5.5 Groundwork and Campus AI OS with the local models (optional), about 1 hour . . . 8

5.6 Multi-GPU scaling on a rented machine (optional) . . . 8

6. Docker, MinIO, GitHub App staging and secrets safety 9

6.1 Secrets rules (read first) . . . 9

6.2 VerifyCloud staging with the official Compose stack and MinIO . . . 9

6.3 Registering a GitHub App for VerifyCloud PR checks . . . 9

7. Native SystemVerilog UVM simulator options and licences 10

8. RISC-V, FPGA and Vivado (and how they differ from CUDA) 11

9. Real Cortex-M and edge-device trials 12

10. Beta programmes: students (Campus AI OS) and RTL engineers (VerifyCloud) 13

11. Deployment, domain, backups and observability 14

12. Multi-GPU rental workflow and cost controls (optional) 15

13. Internship verification and confidentiality (Tata Steel) 16

14. Resume evidence, approvals and interview rehearsal 17

14.1 Review and merge the work . . . 17

14.2 Resume update 17
14.3 Interview rehearsal (per project, 20 minutes each) 17
15. Owner approvals and acceptance signatures **18**

1. One-page prioritized checklist

Do these in order. The first five give the most evidence for the least time.

#	Action	Time	Cost	Unlocks
1	Check the Windows version and run the Windows diagnostic (3.1)	10 min	free	everything below
2	Install WSL2 Ubuntu, the NVIDIA Windows driver, the CUDA toolkit in WSL, and Docker Desktop; then run the GPU diagnostic (3.2–3.6)	1–2 h	free	all GPU steps
3	Run the CUDA kernel benchmarks on the RTX 4060 (5.1)	30 min	free	CUDA kernels: GPU timing moves from BLOCKED to VERIFIED
4	Gather internship records and fix the three resumes (13)	1–2 h	free	honest resume entries
5	Read `master/SCORECARD.md`, check each project's README, then merge the branches (14.1)	1–2 h	free	the work reaches `main`
6	Nebula on a real GPU, and the LLM/InferX runs on the laptop (5.2–5.4)	1 h	free	single-GPU evidence
7	Run the TinyML model on a real Cortex-M board (9)	3–4 h	~₹1,500–3,000 for a Nucleo board	real MCU latency
8	UVM on a real UVM simulator (7)	2–4 h	free with a university licence or EDA Playground	native UVM evidence
9	Stage VerifyCloud, TANDEM and Campus AI OS with real accounts (6, 11)	1 day	domain ~₹800/yr; VM ~$5–12/month	deployment evidence
10	Student and RTL-engineer betas (10)	2–4 weeks	free	real-user evidence
11	Optional: rent multi-GPU time for distributed training and Nebula (5.6)	2–3 h	~$5–20 total	multi-GPU scaling
12	Optional: FPGA bring-up of the RISC-V core (8)	1–2 days	board ~₹6,000–15,000 if not borrowed	FPGA claim on the resume

2. Project-to-owner-step matrix

#	Project	Owner-only steps	Where
1	RaftKV	Merge the branch; publish the LSM C API then run `make test-lsm`; release with a patched Go; optional multi-machine benchmark	14.1; repo `raft-kv/docs/OWNER_ACTIONS.md`
2	LSM Engine	Merge (the on-disk format changed, so migrate any old data); update the Go bindings; real power-loss test on hardware; idle-machine benchmark	14.1; repo `docs/OWNER_ACTIONS.md`
3	TANDEM	Build and run the Docker/Compose stack; deploy with TLS and real secrets; re-measure latency on an idle machine; remove the OT claim from the resume	11, 14.2; repo `docs/OWNER_ACTIONS.md`
4	EMBER	Merge; longer fuzzing runs; idle-machine benchmark	14.1; repo `docs/OWNER_ACTIONS.md`
5	VerifyCloud	Staging with Compose; GitHub App; LLM API key; SMTP; gVisor on worker VMs; choose a maintained S3/MinIO; RTL-engineer beta	6.2, 6.3, 10, 11
6	Vector DB	Idle-machine benchmark; optional real dataset (SIFT1M)	4; `vectordb/docs/`
7	LLM inference engine	No GPU step; optional CPU reproduction	5.4
8	CUDA kernels	**RTX 4060 run** (moves GPU timing from BLOCKED to measured)	3, 5.1
9	Distributed training	RTX 4060 NCCL check; optional multi-GPU rental	5.3, 5.6, 12
10	Groundwork	Full-corpus neural benchmark; larger LLM; choose a licence	5.5
11	RISC-V CPU	FPGA bring-up (or remove the FPGA claim); full ORFS place and route (~15 GB free disk); official CoreMark	8; `riscv-cpu/docs/OWNER_ACTIONS.md`
12	UVM SoC verification	Commercial simulator run (Xcelium, Questa, VCS or EDA Playground)	7
13	INT8/FP8 accelerator	Top-level place and route locally; multi-corner timing; shuttle requirements	`int8-fp8-accelerator/docs/OWNER_ACTIONS.md`
14	Campus AI OS	Deployment; student beta with consent; licence	10, 11
15	MCP Fabric	Real IdP (Keycloak or Auth0); real TLS certificate; clean benchmark	6.1; `mcp-fabric/docs/OWNER_ACTIONS.md`
16	Industrial Edge	**Real Cortex-M board** latency; real vibration data	9
17	InferX	GPU replicas once the engine has a GPU path	5.4
18	Nebula GPU cloud	**RTX 4060 container GPU job**; multi-node	5.2, 5.6
19	EvalOps	GitHub App for check runs; validate a larger judge model	6.3; `evalops/docs/OWNER_ACTIONS.md`
20	EdgeFleet	Real devices (e.g. Raspberry Pi or a Cortex-M board with a bootloader) for the OTA trial	9; `edgefleet/docs/`

Internship and resume steps (13, 14) apply to all projects.

3. Laptop setup: RTX 4060 + WSL2 + CUDA + Docker

Hardware this plan assumes: AMD Ryzen 7, NVIDIA RTX 4060 Laptop GPU (8 GB VRAM, compute capability 8.9, "sm_89"), 16 GB RAM. **Prerequisites:** admin rights on Windows, about 60 GB of free disk on C:, a stable internet connection. **Time:** 1–2 hours. **Cost:** free.

3.1 Confirm the operating system

1. Press **Win + R**, type `winver` and press Enter. **Success:** Windows 11 (any build) or Windows 10 version 21H2 or later. Save a screenshot as `winver.png`.
2. If it is not Windows (for example, you dual-boot Linux), skip 3.2–3.4 and follow the "native Linux" notes in 3.5.
3. Open **PowerShell** (Start → type "PowerShell" → Run as administrator), `cd` to the PROJECTS checkout (or download `master/tools/gpu_diag.ps1` from GitHub), and run:

```
powershell -ExecutionPolicy Bypass -File master\tools\gpu_diag.ps1
```

Success: it writes `gpu_diag_windows_<date>.txt` listing the GPU name and driver version. **Save** that file. **If blocked:** if `nvidia-smi` is "not found", install the driver first (3.3).

3.2 Install WSL2 with Ubuntu 24.04

1. In **PowerShell as administrator**:

```
wsl --install -d Ubuntu-24.04
```

2. Restart Windows when asked. Ubuntu opens and asks for a new Linux username and password. Choose any; this is not your Windows password.
3. In PowerShell: `wsl --update`, then `wsl -l -v`. **Success:** `Ubuntu-24.04` with `VERSION 2`.
4. Give WSL enough memory, but not all of it. Create `%UserProfile%\.wslconfig` (open Notepad, paste the text below, and use **File → Save As** with "All files", named `.wslconfig` in `C:\Users\<you>\`):

```
[wsl2]
memory=10GB
swap=8GB
processors=8
```

5. Run `wsl --shutdown` in PowerShell, then reopen Ubuntu. **If blocked:** "Virtualization not enabled" means you must reboot into the BIOS/UEFI (usually F2 or Del at boot) and enable **SVM Mode** (AMD) or **Virtualization**. Then repeat step 1.

3.3 Install the NVIDIA Windows driver (the only GPU driver you need)

1. Open https://www.nvidia.com/Download/index.aspx. Choose **GeForce → GeForce RTX 40 Series (Notebooks) → GeForce RTX 4060 Laptop GPU → Windows 11** (or 10) → **Studio Driver** (more stable) → Search → Download. You can use the NVIDIA App instead.
2. Run the installer and choose **Custom → Clean installation**. Reboot.
3. **Do not install any NVIDIA driver inside Ubuntu.** WSL uses the Windows driver through `/usr/lib/wsl/lib`.
4. In Ubuntu: `nvidia-smi`. **Success:** a table showing "NVIDIA GeForce RTX 4060 Laptop GPU", 8188 MiB and a CUDA version of 12.x or later. **Save** the output as `nvidia-smi.txt`.

3.4 Install the CUDA toolkit inside WSL (compiler, libraries)

```
# WSL
cd ~
wget https://developer.download.nvidia.com/compute/cuda/repos/wsl-ubuntu/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt-get update
sudo apt-get install -y cuda-toolkit-12-8 build-essential git cmake python3-venv
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
source ~/.bashrc
nvcc --version
```

Success: `nvcc` reports release 12.8. **Why 12.x:** the CUDA kernels project links `libcudart.so.12`. **If blocked:** if apt cannot find `cuda-toolkit-12-8`, run `apt-cache search cuda-toolkit-12` and install the newest 12.x listed.

3.5 Install Docker Desktop with GPU support

1. Download Docker Desktop for Windows from https://www.docker.com/products/docker-desktop/ and install it with "Use WSL 2 instead of Hyper-V" ticked.
2. Open Docker Desktop → **Settings (gear) → Resources → WSL integration** → turn on **Ubuntu-24.04 → Apply & restart**.
3. In Ubuntu:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi -L
```

Success: `GPU 0: NVIDIA GeForce RTX 4060 Laptop GPU (UUID: …)`. **If blocked:** update Docker Desktop and the NVIDIA driver, then `wsl --shutdown`. Native Linux: install the NVIDIA driver from your distribution, plus `nvidia-container-toolkit`, then run `sudo nvidia-ctk runtime configure --runtime=docker && sudo systemctl restart docker`.

3.6 Get the code and run the one-command diagnostic

```
# WSL — keep code inside the Linux filesystem (~/src), not /mnt/c (10x slower)
mkdir -p ~/src && cd ~/src
git clone https://github.com/siddhaantxsingh/PROJECTS.git
cd PROJECTS && git checkout ccr-a6e6c567-y4w7k8   # or main after you merge
bash master/tools/gpu_diag.sh
```

Success: a file `gpu_diag_<date>.txt` containing the GPU name, driver, `compute_cap 8.9`, 8188 MiB, power limit and temperature, plus `nvcc` and Docker versions. **Save it** in `master/evidence/owner/`.

3.7 Rules for every GPU measurement

- Plug in the charger. Set Windows **Settings → System → Power → Power mode → Best performance**. Close games, browsers with video, and anything else using the GPU.
- Run one benchmark at a time. Wait 2 minutes between runs so the GPU cools.
- Record power, clocks and temperature during every run, in a second Ubuntu terminal:

```
nvidia-smi dmon -s pucm -d 1 -o T -f gpu_dmon_$(date +%H%M%S).log
```

- Run each benchmark 3 times. Keep all three raw outputs, not just the best one.
- Laptop GPUs throttle. If the temperature stays above 85 °C or the clocks drop, note it next to the result.

4. Benchmark evidence capture (applies to every project)

1. Before each session, save `gpu_diag_<date>.txt` (3.6) and `git rev-parse HEAD`.
2. Run the exact command the project documents. Pipe everything to a file: `cmd 2>&1 | tee docs/evidence/owner/<name>_<date>_run1.log`.
3. Keep the raw logs, the `nvidia-smi dmon` log, and a short `README.md` in the same folder that says: date, machine, commit, command, and anything unusual.
4. Only after that, update the project's benchmark table, and link the raw log from each number.
5. Commit with a message like "evidence: RTX 4060 run of gpu_bench (3 runs)". **Never** edit numbers by hand, round them up, or drop a slow run.

5. GPU and laptop runs, project by project

Do these after section 3, one at a time. Every script below exists in the repository and was written by the audit. **None of them has ever run on a GPU**, because the build machine had no GPU. Treat their first run as the real test. If a step fails, save the output anyway: a recorded failure is evidence, and it tells the next session what to fix.

5.1 CUDA kernels (RTX 4060, sm_89): about 30 minutes

```
# WSL
cd ~/src/PROJECTS/cuda-kernels
sudo apt-get install -y build-essential g++-13 clang-18 python3-pip
tools/owner_rtx4060.sh
```

- **What it does:**
  - records the GPU, driver and clocks;
  - runs the 150 emulator checks;
  - builds `bench/gpu_bench.cu` for sm_89 (with `nvcc` if installed, otherwise clang 18 and the PyPI toolkit, as in CI);
  - runs the benchmark 3 times while `nvidia-smi dmon` records.
- **Success:**
  - `make_test.log` ends with `150 checks, 0 failures`;
  - each `runN.txt` ends with `0 correctness check failures`;
  - the `mma.m16n8k16 fragment layout (probe)` line says PASS. That proves the emulator's tensor-core model matches real hardware.
- **Save:** commit `cuda-kernels/results/gpu/<date>-rtx4060/`. Then change the GPU-timing rows in `docs/requirements.md` from BLOCKED to measured, quoting the median of the 3 runs plus the clocks and power from `dmon.txt`.
- **If blocked:**
  - **Build error:** install the CUDA toolkit (3.4), so the script uses `nvcc -arch=sm_89`.
  - **A `check` FAIL:** do not publish any GFLOP/s numbers. Keep the folder and open an issue with it.

5.2 Nebula GPU cloud: a real GPU job in a container, about 30 minutes

```
# WSL (Docker Desktop with GPU support from 3.5)
docker run --rm --gpus all ubuntu:24.04 nvidia-smi        # must print the RTX 4060
cd ~/src/PROJECTS/nebula-gpu-cloud
cargo build --release
scripts/owner_gpu_check.sh
```

- **Success:**
  - the node registers with `"model":"NVIDIA GeForce RTX 4060"`, plus its memory and UUID;
  - a job runs in a container and prints `GPU 0: NVIDIA GeForce RTX 4060`, with `CUDA_VISIBLE_DEVICES=0`;
  - the script ends with `PASS: real GPU job ran in a container`.
- **Save:**
  - commit `docs/evidence/owner-gpu-<date>/`;
  - copy the real `nvidia-smi` line into the `ONE_4060` fixture in `src/gpu.rs`, then run `cargo test --release gpu::`.
- **If blocked:**
  - a permissions error inside the container: retry with `--container-user` set to a uid in the `video` group, and note it in the evidence;
  - if Docker cannot see the GPU, go back to 3.5.

5.3 Distributed training: single-GPU NCCL check, about 20 minutes

```
cd ~/src/PROJECTS/distributed-training
bash owner/rtx4060_single_gpu.sh
```

- **What it does:**
  - installs CUDA PyTorch (cu126);
  - runs a one-rank NCCL sanity check;
  - runs a one-rank CUDA DDP training run in float64, compared with dtrain's CPU result.

- **Success:**
  - `"nccl_sanity_ok": true`;
  - parameter differences per step of about 1e-15 to 1e-12. This is not exactly 0, because GPU kernels add numbers in a different order.
  - `8 passed` from pytest.
- **Save:** the files the script names, in `benchmarks/results/<date>/gpu/`.
- **Know the limit: one GPU cannot show multi-GPU scaling**, which needs 5.6. dtrain itself is NumPy/CPU; this checks PyTorch on CUDA against it.

5.4 LLM inference engine and InferX: no GPU step

- The engine has **no GPU code path**, so there is nothing to run on the RTX 4060. Do not describe it as a GPU engine.
- Optional: reproduce the CPU results on the laptop with the commands in `llm-inference-engine/docs/OWNER_ACTIONS.md`. The expected chat answer is "The capital of France is Paris."
- **InferX:** its GPU replicas stay BLOCKED until the engine gains a GPU path. The CPU benchmarks are in `inferx/docs/`.

5.5 Groundwork and Campus AI OS with the local models (optional), about 1 hour

The local GGUF models run on CPU. On the laptop they will be faster than on the build machine.

1. `python3 groundwork/scripts/fetch_models.py` downloads the four Apache-2.0 models from Docker Hub (about 2.7 GB).
2. Run the full-corpus neural benchmark and the reranker-quality run, using `groundwork/docs/OWNER_ACTIONS.md`.

- **Success:** NDCG@10 and recall numbers for the **full** corpus. The audit ran only a 31-document sub-corpus.
- **Save:** the new results folder.

5.6 Multi-GPU scaling on a rented machine (optional)

Follow section 12 for cost controls first. On a 2× or 4× GPU instance:

```
sudo shutdown -h +120          # hard stop after 2 hours, set FIRST
git clone https://github.com/siddhaantxsingh/PROJECTS.git && cd PROJECTS
git checkout ccr-a6e6c567-y4w7k8
nvidia-smi topo -m > topo.txt
bash distributed-training/owner/multi_gpu_nccl.sh
```

- **Success:**
  - NCCL all-reduce bandwidth for 1/2/4 GPUs;
  - DDP throughput scaling;
  - the parameter cross-check still within about 1e-12.
- **Save:** all output files and `topo.txt`.
- **Nebula multi-node:** run two agents on two GPU machines, using the same steps as 5.2.

6. Docker, MinIO, GitHub App staging and secrets safety

6.1 Secrets rules (read first)

- Secrets (API keys, private keys, OAuth client secrets, webhook secrets, database passwords) go **only** in `.env` files that git ignores, or in your password manager. Check this with `git check-ignore -v .env`; it must print a rule.
- Generate strong values locally: `openssl rand -hex 32` (or `openssl rand -base64 32` where a project asks for base64).
- Never paste a secret into a chat, an issue, a README, a screenshot or a log you commit. If one leaks: revoke it at the provider **first**, then remove it from git history.
- Turn on GitHub **Settings → Code security → Secret scanning** and **Push protection** for each repository (free on public repos).

6.2 VerifyCloud staging with the official Compose stack and MinIO

1. In WSL: `cd ~/src && git clone https://github.com/siddhaantxsingh/VerifyCloud.git && cd VerifyCloud/verifycloud && git checkout ccr-a6e6c567-y4w7k8`.
2. `cp .env.example .env` and fill every empty value using 6.1. Do not change the image tags.
3. `docker compose up -d --build --wait`. **Success:** `docker compose ps` shows every service "healthy".
4. Run the end-to-end tests named in `docs/OWNER_ACTIONS.md` of that repo. Save the output.
5. Stop with `docker compose down`. Add `-v` **only** if you want to delete the data volumes.

6.3 Registering a GitHub App for VerifyCloud PR checks

1. GitHub → your avatar → **Settings → Developer settings → GitHub Apps → New GitHub App**.
2. Name it, for example "verifycloud-dev-siddhant". Homepage URL: your repository URL. Webhook URL: the public HTTPS URL of your staging server (see 11), ending in the path the repo's docs give. Webhook secret: `openssl rand -hex 32`, saved into `.env`.
3. Permissions: **Checks: Read & write**, **Commit statuses: Read & write**, **Contents: Read**, **Pull requests: Read**, **Metadata: Read**. Subscribe to events: **Pull request**, **Push**, **Check suite**.
4. **Create** → **Generate a private key** (it downloads a `.pem`) → move it next to `.env` (git-ignored) → set the path and App ID in `.env`.
5. **Install App** → choose **Only select repositories** → pick a throwaway test repository, not your real projects.
6. Open a PR in the test repository. **Success:** a "VerifyCloud" check appears on the PR. **Save** a screenshot and the webhook delivery log (App → **Advanced → Recent deliveries**).
7. **If blocked:** a red X on the webhook delivery means the URL is not reachable from the internet. Check DNS, TLS and the firewall (11).

7. Native SystemVerilog UVM simulator options and licences

The UVM project runs today on **pyuvm + cocotb + Verilator**, which is real UVM methodology in Python. The directive also asks for evidence from a **native SystemVerilog UVM** simulator. Options, cheapest first:

Option	Licence	How to get it	Notes
University EDA licence (Cadence Xcelium, Siemens Questa or Synopsys VCS)	Academic licence from your department	Ask the VLSI lab admin. Your matrix-multiplier work used Cadence Genus and Innovus, so a Cadence licence probably exists, and Xcelium may be included	Best option. Run on a lab machine; never copy licence files off it
EDA Playground (edaplayground.com)	Free account; commercial simulators need a verified login	Sign up → verify email → choose "Aldec Riviera-PRO" or "Synopsys VCS" with UVM 1.2	Good for small runs; paste files, keep run logs
Verilator 5.x with UVM support	Free (open source)	Recent Verilator releases have growing UVM support	See the project's own docs for what was tried here

Steps on a lab machine with Xcelium (adapt the file list from the project's `docs/OWNER_ACTIONS.md`):

```
xrun -uvm -sv -access +r -f sv_uvm/files.f +UVM_TESTNAME=soc_smoke_test -log xrun_smoke.log
```

Success: the log ends with `UVM_ERROR : 0` and `UVM_FATAL : 0` and the test report. **Save** the log and the coverage report (`imc` or `urg` output). **If blocked:** for a licence checkout error, ask the lab admin. Never use cracked tools; the evidence would be worthless and possibly illegal.

8. RISC-V, FPGA and Vivado (and how they differ from CUDA)

- **Vivado is AMD's FPGA tool.** It turns RTL into a bitstream for a Xilinx/AMD FPGA board. It has nothing to do with NVIDIA, CUDA or your RTX 4060. A GPU cannot "run" Verilog.
- The RISC-V core and the accelerator are verified here in simulation (Verilator, Spike), and taken through synthesis and place-and-route on open PDKs (OpenROAD). None of that is an FPGA run.
- The Electronics resume says the RISC-V core was "deployed on FPGA hardware". If that is not true for this core, remove it (see `master/RESUME_EVIDENCE.md`), or do the bring-up below.

FPGA bring-up (optional, 1–2 days):

1. Borrow or buy a board with enough LUTs and a USB-UART, for example Digilent Basys 3 or Arty A7-35T.
2. Install **Vivado ML Standard** (free) from amd.com → Support → Downloads, on Windows. It needs ~60–90 GB. Pick only the Artix-7 device family to save space.
3. Create a project, add the files in `riscv-cpu/rtl/`, add a top wrapper that connects the clock and reset, LEDs and UART, and a constraints `.xdc` file for your board (Digilent publishes master XDC files).
4. Run Synthesis → Implementation → **Report Timing Summary**. **Save** the timing report (WNS must be ≥ 0 at your clock) and the utilization report.
5. Program the board (**Open Hardware Manager → Program device**). Run a test program that prints over UART. **Save** a photo, the UART log, and the reports.
6. **If blocked:** for a negative WNS, lower the clock (use the Clocking Wizard to make 25 MHz, for example). For a missing UART, check the board's UART pins in the XDC file.

9. Real Cortex-M and edge-device trials

Goal: replace the QEMU estimates in Industrial Edge (and EdgeFleet's simulated devices) with real board measurements.

1. **Buy or borrow** an STM32 Nucleo board with a Cortex-M4F, for example NUCLEO-F446RE or NUCLEO-L476RG. These have an on-board ST-LINK, so you need no extra programmer.
2. Install the tools in WSL: `sudo apt-get install -y gcc-arm-none-eabi openocd`. On Windows, also install **STM32CubeProgrammer** (st.com, free, needs an account) as a fallback flasher.
3. Pass the board's USB through to WSL: on Windows, install `usbipd-win` (`winget install usbipd`), then in PowerShell (admin) run `usbipd list` → `usbipd bind --busid <id>` → `usbipd attach --wsl --busid <id>`.
4. Build and flash using the commands in `industrial-edge/docs/OWNER_ACTIONS.md`. That file was written by the audit and gives the exact target, linker script and UART settings.
5. Open the serial console: `sudo apt-get install -y picocom && picocom -b 115200 /dev/ttyACM0`. **Success:** the firmware prints per-inference cycle counts from the DWT cycle counter, and test-set accuracy that matches the QEMU run bit for bit.
6. **Save:** the UART log, a photo of the board, the `arm-none-eabi-size` output and the board clock. Then fill the "real board" column in the benchmarks doc.
7. **If blocked:** for "No ST-LINK detected", repeat step 3. For a hard fault, check that the linker script matches your MCU's flash and RAM sizes.

10. Beta programmes: students (Campus AI OS) and RTL engineers (VerifyCloud)

Ethics and privacy first:

- Write a one-page consent note: what the tool does, what is logged, how long it is kept, and how to ask for deletion.
- Collect no personal data beyond an email address for login.
- Do not upload copyrighted course material without the instructor's permission.
- Get your department's approval before running a study with students.

1. **Recruit.** Campus AI OS: 10–20 students from one course, with the instructor's agreement. VerifyCloud: 5–10 RTL engineers or students from the VLSI lab or an online RTL community. Post a short message saying what it is, how long it takes (30 minutes), and that it is a student project.
2. **Prepare.** Use the staging deployment (11). Create accounts by invitation only, and have a feedback form ready (Google Forms is fine) with 5 questions: task success, time taken, a 1–5 rating, the worst problem, and a free comment.
3. **Run** for 2 weeks. Record usage metrics from the app's metrics endpoint (counts only, no content).
4. **Save:** anonymised metrics (CSV), aggregated form results, and a one-page summary of what you changed because of the feedback. **Never** invent users or quotes. If fewer people join than hoped, report the real number.

11. Deployment, domain, backups and observability

Minimal, cheap and honest setup for TANDEM, Campus AI OS, VerifyCloud and MCP Fabric staging:

1. **Domain:** buy one from any registrar (~₹800–1,200 a year). Use subdomains such as `tandem.<domain>` and `campus.<domain>`.
2. **Server:** one small Linux VM with 2 vCPU and 4 GB of RAM (~$5–12 a month at common providers). Set a **billing alert** at your chosen limit in the provider console before creating the VM.
3. **Harden** the server:
   - create a non-root user with an SSH key;
   - disable password login;
   - turn on automatic security updates (`sudo apt install unattended-upgrades`);
   - firewall: `sudo ufw allow OpenSSH; sudo ufw allow 80,443/tcp; sudo ufw enable`.
4. **Deploy** with each project's documented `docker compose` file. Put a reverse proxy in front with automatic TLS; Caddy is the simplest: one line per site, e.g. `tandem.<domain> { reverse_proxy localhost:8080 }`.
5. **Backups:**
   - a nightly `pg_dump`/volume snapshot to a separate storage bucket;
   - **test a restore** into a fresh container once, and save the log;
   - keep 7 daily and 4 weekly copies.
6. **Observability:**
   - a free uptime monitor hitting each `/healthz`;
   - the projects' Prometheus `/metrics` endpoints, scraped by a small Prometheus + Grafana stack, or at least checked with curl;
   - logs kept for 14 days.
7. **Save:** the deploy logs, a screenshot of the TLS certificate, the restore-test log, and an uptime report after 1 week.

12. Multi-GPU rental workflow and cost controls (optional)

See 5.6 for what to run. This section is about not overspending.

1. Pick a provider that bills by the second or minute and offers 2× or 4× GPU instances with NCCL, e.g. a cloud GPU marketplace or a major cloud. **Check current prices yourself;** they change often.
2. **Before starting:** set a prepaid credit limit or a billing alert of, for example, $20. Write down the expected cost: hourly price × planned hours.
3. Start the instance with an image that has CUDA and Docker. Run `nvidia-smi topo -m` and save it, because it shows whether NVLink or PCIe connects the GPUs.
4. Run the commands from 5.6 with a hard time limit: `sudo shutdown -h +120` as the very first command, so the machine powers off after 2 hours even if you forget.
5. Copy the logs back with `scp`. **Terminate** (not just stop) the instance and **delete attached volumes**. Check the billing page the next day.

13. Internship verification and confidentiality (Tata Steel)

Full analysis: `master/RESUME_EVIDENCE.md`. Steps:

1. **Collect:** the offer letter, the experience certificate (title and dates), the stipend record, and the name and email of your manager or mentor. Store them privately, never in a public repo.
2. **Fix the title:** use the exact certificate title on all three resumes.
3. **Check the facts with your manager** (a 10-minute call or email): MCU part number, model type, how latency was measured, document count, whether the RAG tool was used by engineers, and any before/after numbers.
4. **Permission:** ask in writing whether you may describe the work on your resume and in interviews at the level of the bullets you plan to use. Ask whether you may say "Tata Steel".
5. **Confidentiality:** never publish SOP content, plant data, internal screenshots, code or architecture diagrams. The personal projects (Industrial Edge, Groundwork, Campus AI OS) use synthetic or public data and must say so.
6. **Write the bullets** from the template in RESUME_EVIDENCE.md section 1.4, using only the facts confirmed in step 3.
7. **Never** backdate commits, put the company name in personal repositories, or describe a personal project as an internship deliverable.

14. Resume evidence, approvals and interview rehearsal

14.1 Review and merge the work

1. On GitHub, open each repository → **Branches** → `ccr-a6e6c567-y4w7k8` → **Compare & pull request** (you open the PR, not the agent). Read the diff summary and the project's `docs/AUDIT_2026-10-14.md` or `docs/requirements.md`.
2. Check that the **Actions** tab shows green for the branch.
3. Merge when satisfied. **Rule:** merge only what you understand well enough to explain in an interview.

14.2 Resume update

- Use only bullets from `master/SCORECARD.md` ("Supported resume bullets"). Each links to its evidence file.
- Remove or rewrite every claim marked NOT SUPPORTED in `master/RESUME_EVIDENCE.md`: the Kubernetes PaaS, LoRA fine-tuning, the FPGA deployment, and the 1M-vector figure unless you re-run it.
- Make the shared items (open-source PRs, rank, dates) say the same thing in all three resumes.

14.3 Interview rehearsal (per project, 20 minutes each)

1. **Two-minute walkthrough:** problem → design → the hardest bug → evidence → what is not done.
2. **Three "why" questions:** why this design over the obvious alternative (use the ADRs); what breaks first under load; how you know it is correct (which test, which invariant).
3. **One live demo** you can run in under 3 minutes from a fresh clone. Practise it twice.
4. **The honesty question:** "What on your resume would you not want me to dig into?" There should be no answer. If there is, fix the resume.
5. Each project's `docs/INTERVIEW.md` (where present) lists likely questions.

15. Owner approvals and acceptance signatures

Tick each box only after you have done the check yourself. Sign and date the bottom of the page and keep it with your records.

☐	Project	I ran or reviewed	Evidence file I checked	Initials / date
☐	RaftKV			
☐	LSM Engine			
☐	TANDEM			
☐	EMBER			
☐	VerifyCloud			
☐	Vector DB			
☐	LLM inference engine			
☐	CUDA kernels			
☐	Distributed training			
☐	Groundwork			
☐	RISC-V CPU			
☐	UVM SoC verification			
☐	INT8/FP8 accelerator			
☐	Campus AI OS			
☐	MCP Fabric			
☐	Industrial Edge			
☐	InferX			
☐	Nebula GPU cloud			
☐	EvalOps			
☐	EdgeFleet			

Signature: _____ **Date:** _____ — I have reviewed the evidence for each ticked project, and I can explain every claim I will make about it.